

Core Language Working Group "ready" Issues for the February, 2016 (Kona) meeting

Section references in this document reflect the section numbering of document [WG21 N4606](#).

1622. Empty aggregate initializer for union

Section: 8.6.1 [dcl.init.aggr] **Status:** ready **Submitter:** Daveed Vandevoorde **Date:** 2013-02-14

According to 8.6.1 [dcl.init.aggr] paragraph 15,

When a union is initialized with a brace-enclosed initializer, the braces shall only contain an *initializer-clause* for the first non-static data member of the union.

This would appear to preclude using {} as the initializer for a union, which would otherwise have reasonable semantics. Is there a reason for this restriction?

Also, paragraph 7 reads,

If there are fewer *initializer-clauses* in the list than there are members in the aggregate, then each member not explicitly initialized shall be initialized from an empty initializer list (8.6.4 [dcl.init.list]).

There should presumably be special treatment for unions, so that only a single member is initialized in such cases.

(See also issue 1460.)

Proposed resolution (November, 2016):

Change 8.6.1 [dcl.init.aggr] paragraph 8 as follows:

~~If there are fewer *initializer-clauses* in the list than there are elements in the aggregate, then each~~ **Each non-variant**
element **of the aggregate that is** not explicitly initialized ~~shall be~~ **is** initialized from its default member initializer (9.2 [class.mem]) or, if there is no default member initializer, **copy-initialized** from an empty initializer list (8.6.4 [dcl.init.list]). **If the aggregate is a union and the initializer list is empty, then**

- if any union member has a default member initializer, that member is initialized from its default member initializer;**
- otherwise, the first member of the union (if any) is copy-initialized from an empty initializer list.**

[Example:...

1677. Constant initialization via aggregate initialization

Section: 3.6.2 [basic.start.static] **Status:** ready **Submitter:** Daniel Krügler **Date:** 2013-05-05

The resolution of issue 1489 added wording regarding value initialization to 3.6.2 [basic.start.static] paragraph 2 in an attempt to clarify the status of an example like

```
int a[1000]{};
```

However, this example is aggregate initialization, not value initialization. Also, now that we allow *brace-or-equal-initializers* in aggregates, this wording also needs to be updated to allow an aggregate with constant non-static data member initializers to qualify for constant initialization.

Proposed resolution (November, 2017):

1. Change 3.6.2 [basic.start.static] paragraph 2 as follows, converting the bulleted list into running text as indicated:

A **constant initializer** for an **a variable or temporary** object *o* is an **expression that** **initializer whose full-expression** is a constant expression, except that **if *o* is an object, such an initializer** may also invoke `constexpr` constructors for *o* and its subobjects even if those objects are of non-literal class types. [Note: Such a class may have a non-trivial destructor —end note] *Constant initialization* is performed:

- **if each full-expression (including implicit conversions) that appears in the initializer of a reference with static or thread storage duration is a constant expression (5.20 [expr.const]) and the reference is bound to a glvalue designating an object with static storage duration, to a temporary object (see 12.2 [class.temporary]) or subobject thereof, or to a function;**
- **if an a variable or temporary** object with static or thread storage duration is initialized by a **constructor call, and if the initialization full-expression is a** constant initializer for the **object;** **entity**
- **if an object with static or thread storage duration is not initialized by a constructor call and if either the object is value-initialized or every full-expression that appears in its initializer is a constant expression.**

If constant initialization is not performed...

2. Change 5.20 [expr.const] paragraph 2 as follows:

~~A conditional-expression~~ **An expression** *e* is a *core constant expression* unless...

1860. What is a “direct member?”

Section: 9.3 [class.union] **Status:** ready **Submitter:** Dawn Perchik **Date:** 2014-02-13

The term “direct member” is used in 9.3 [class.union] paragraph 8 but is not defined. It might be better to refer to the class's *member-specification* instead.

Additional note, October, 2015:

This issue is expected to be addressed by the wording of N4532 or a successor thereof (“Default Comparisons”).

Proposed resolution (November, 2016):

1. Change 8.6.1 [dcl.init.aggr] paragraph 2 as follows:

The *elements* of an aggregate are:

- for an array, the array elements in increasing subscript order, or
- for a class, the direct base classes in declaration order followed by the direct **non-static data** members **that are not members of an anonymous union,** in declaration order.

2. Change 9.2 [class.mem] paragraph 1 as follows:

The *member-specification* in a class definition declares the full set of members of the class; no member can be added elsewhere. **A direct member of a class *x* is a member of *x* that was first declared within the member-specification of *x*, including anonymous union objects and direct members thereof.** Members of a class are...

3. Change 9.3.1 [class.union.anon] paragraph 1 as follows:

A union of the form

```
union { member-specification } ;
```

is called an *anonymous union*; it defines an unnamed **type and an unnamed** object of **unnamed** **that** type **called an anonymous union object**. Each *member-declaration* in the *member-specification* of an anonymous union

2174. Unclear rules for friend definitions in templates

Section: 14.5.4 [temp.friend] **Status:** ready **Submitter:** Richard Smith **Date:** 2015-09-17

According to 14.5.4 [temp.friend] paragraph 4,

When a function is defined in a friend function declaration in a class template, the function is instantiated when the function is odr-used (3.2 [basic.def.odr]). The same restrictions on multiple declarations and definitions that apply to non-template function declarations and definitions also apply to these implicit definitions.

This seems to imply that:

1. Instantiating a class template that contains a friend function definition instantiates the declaration, but not the definition, of that friend function, as usual (but see below).
2. If the function is odr-used, a definition is instantiated for each such class template specialization whose template had a definition.
3. If that results in multiple definitions, the program is ill-formed as usual.

The intent appears to be that the instantiated friend function declarations should be treated as if they were definitions, but that's not clear from the wording. This wording is also missing similar provisions for friend function template definitions; there is implementation divergence on the treatment of such cases.

There also does not appear to be wording that says that instantiating a class template specialization results in the instantiation of friend functions declared/defined therein (the relevant wording was removed from this section by issue 329). Presumably this should be covered in 14.7.1 [temp.inst] paragraph 1, which also includes the following wording that could be reused for the friend case:

However, for the purpose of determining whether an instantiated redeclaration of a member is valid according to 9.2 [class.mem], a declaration that corresponds to a definition in the template is considered to be a definition.

Also, the reliance on odr-use to trigger friend instantiation is out of date, as there are other contexts that can require an instantiation when there is no odr-use (a `constexpr` function invoked within an unevaluated operand).

Proposed resolution (October, 2015) [SUPERSEDED]:

1. Delete 14.5.4 [temp.friend] paragraph 4:

~~When a function is defined in a friend function declaration in a class template, the function is instantiated when the function is odr-used (3.2 [basic.def.odr]). The same restrictions on multiple declarations and definitions that apply to non-template function declarations and definitions also apply to these implicit definitions.~~

2. Change 14.7.1 [temp.inst] paragraph 1 as follows:

...The implicit instantiation of a class template specialization causes the implicit instantiation of the declarations, but not of the definitions, default arguments, or *exception-specifications* of the class member functions, member classes, scoped member enumerations, static data members, **and** member templates, **and friends**; and it causes the implicit instantiation of the definitions of unscoped member enumerations and member anonymous unions. However, for the purpose of determining whether an instantiated redeclaration **of a member** is valid according to **3.2 [basic.def.odr] and** 9.2 [class.mem], a declaration that corresponds to a definition in the template is considered to be a definition. [Example:...

3. Change 14.7.1 [temp.inst] paragraph 3 as follows:

Unless a function template specialization has been explicitly instantiated or explicitly specialized, the function template specialization is implicitly instantiated when the specialization is referenced in a context that requires a function definition to exist. **A function whose declaration was instantiated from a friend function definition is implicitly instantiated when it is referenced in a context that requires a function definition to exist.** Unless a call is to a function template explicit specialization or to a member function of an explicitly specialized class template, a default argument for a function template or a member function of a class template is implicitly instantiated when the function is called in a context that requires the value of the default argument.

Proposed resolution (November, 2016):

1. Delete 14.5.4 [temp.friend] paragraph 4:

When a function is defined in a friend function declaration in a class template, the function is instantiated when the function is odr-used (3.2 [basic.def.odr]). The same restrictions on multiple declarations and definitions that apply to non-template function declarations and definitions also apply to these implicit definitions.

2. Change 14.7.1 [temp.inst] paragraph 1 as follows, splitting it into two paragraphs as indicated:

... [Note: Within a template declaration, a local class (9.4 [class.local]) or enumeration and the members of a local class are never considered to be entities that can be separately instantiated (this includes their default arguments, *exception-specifications*, and non-static data member initializers, if any). As a result, the dependent names are looked up, the semantic constraints are checked, and any templates used are instantiated as part of the instantiation of the entity within which the local class or enumeration is declared. —end note]

The implicit instantiation of a class template specialization causes the implicit instantiation of the declarations, but not of the definitions, default arguments, or *exception-specifications* of the class member functions, member classes, scoped member enumerations, static data members, and member templates, and friends; and it causes the implicit instantiation of the definitions of unscoped member enumerations and member anonymous unions. However, for the purpose of determining whether an instantiated redeclaration of a member is valid according to 3.2 [basic.def.odr] and 9.2 [class.mem], a declaration that corresponds to a definition in the template is considered to be a definition. [Example:

```
template<class T, class U>
struct Outer {
    template<class X, class Y> struct Inner;
    template<class Y> struct Inner<T, Y>; // #1a
    template<class Y> struct Inner<T, Y> { }; // #1b; OK: valid redeclaration of #1a
    template<class Y> struct Inner<U, Y> { }; // #2
};

Outer<int, int> outer; // error at #2
```

Outer<int, int>::Inner<int, Y> is redeclared at #1b. (It is not defined but noted as being associated with a definition in Outer<T, U>.) #2 is also a redeclaration of #1a. It is noted as associated with a definition, so it is an invalid redeclaration of the same partial specialization.

```
template<typename T> struct Friendly {
    template<typename U> friend int f(U) { return sizeof(T); }
};
Friendly<char> fc;
Friendly<float> ff; // ill-formed: produces second definition of f(U)
```

—end example]

3. Change 14.7.1 [temp.inst] paragraph 3 as follows:

Unless a function template specialization has been explicitly instantiated or explicitly specialized, the function template specialization is implicitly instantiated when the specialization is referenced in a context that requires a function definition to exist. A function whose declaration was instantiated from a friend function definition is implicitly instantiated when it is referenced in a context that requires a function definition to exist. Unless a call is to a function template explicit specialization or to a member function of an explicitly specialized class template, a default argument for a function template or a member function of a class template is implicitly instantiated when the function is called in a context that requires the value of the default argument.

2191. Incorrect result for noexcept(typeid(v))

Section: 15.4 [except.spec] Status: ready Submitter: Mike Miller Date: 2015-10-26

Consider the following example:

```
struct B { virtual void f() { } };
struct D : B { } d;
bool b = noexcept(typeid(d));
```

According to 5.3.7 [expr.unary.noexcept] paragraph 3, the value of `b` should be `false`, because 15.4 [except.spec] bullet 14.6 says,

- If `e` is a `typeid` expression applied to a glvalue expression whose type is a polymorphic class type (5.2.8 [expr.typeid]), `S` consists of the type `std::bad_typeid`.

and `d` is such an expression. This is clearly bogus, as the expression cannot possibly throw; according to 5.2.8 [expr.typeid] paragraph 2, the condition under which the exception might be thrown is:

If the glvalue expression is obtained by applying the unary `*` operator to a pointer⁶⁹ and the pointer is a null pointer value (4.11 [conv.ptr]), the `typeid` expression throws an exception (15.1 [except.throw]) of a type that would match a handler of type `std::bad_typeid` exception (18.7.4 [bad.typeid]).

Proposed resolution (November, 2016):

Change 15.4 [except.spec] bullet 13.6 as follows:

The *set of potential exceptions of an expression* `e` is empty if `e` is a core constant expression (5.20 [expr.const]). Otherwise, it is the union of the sets of potential exceptions of the immediate subexpressions of `e`, including default argument expressions used in a function call, combined with a set `S` defined by the form of `e`, as follows:

- ...
- If `e` is a `typeid` expression applied to a glvalue expression whose type is (possibly parenthesized) built-in unary `*` operator applied to a pointer to a polymorphic class type (5.2.8 [expr.typeid]), `S` consists of the type `std::bad_typeid`. [Example: ...

2205. Restrictions on use of `alignas`

Section: 7.6.1 [dcl.attr.grammar] **Status:** ready **Submitter:** Richard Smith **Date:** 2015-11-30

According to 7.6.1 [dcl.attr.grammar] paragraph 5, a program is ill-formed if an *attribute* appertains to an entity or statement to which it is not allowed to apply. Presumably an *alignment-specifier* should have the same restriction.

Proposed resolution (November, 2016):

Change 7.6.1 [dcl.attr.grammar] paragraph 5 as follows:

Each *attribute-specifier-seq* is said to *appertain* to some entity or statement, identified by the syntactic context where it appears (Clause 6 [stmt.stmt], Clause 7 [dcl.dcl], Clause 8 [dcl.decl]). If an *attribute-specifier-seq* that appertains to some entity or statement contains an *attribute* or *alignment-specifier* that is not allowed to apply to that entity or statement, the program is ill-formed. If an *attribute-specifier-seq* appertains to a friend declaration (11.3 [class.friend]), that declaration shall be a definition. No *attribute-specifier-seq* shall appertain to an explicit instantiation (14.7.2 [temp.explicit]).

2218. Ambiguity and namespace aliases

Section: 3.4 [basic.lookup] **Status:** ready **Submitter:** Richard Smith **Date:** 2015-12-29

There is implementation divergence on the status of the following example:

```
namespace A { namespace B { int x; } }
namespace C { namespace B = A::B; }
using namespace A;
using namespace C;
int x = B::x;
```

This should presumably be valid: the lookup of `B` finds `A::B` and `C::B`, but it is not ambiguous because they denote the same entity. A similar example with a *using-declaration* or *alias-declaration* seems to be universally accepted. Perhaps the lookup rules need to be clarified regarding the status of this example.

Proposed resolution (November, 2016):

Change 3.4 [basic.lookup] paragraph 1 as follows:

The name lookup rules apply uniformly to all names (including *typedef-names* (7.1.3 [dcl.typedef]), *namespace-names* (7.3 [basic.namespace]), and *class-names* (9.1 [class.name])) wherever the grammar allows such names in the context discussed by a particular rule. Name lookup associates the use of a name with a **declaration** **set of declarations** (3.1 [basic.def]) of that name. ~~Name lookup shall find an unambiguous declaration for the name (see 10.2 [class.member.lookup]).~~ ~~Name lookup may associate more than one declaration with a name if it finds the name to be a function name;~~ **The declarations found by name lookup shall either all declare the same entity or shall all declare functions; in the latter case,** the declarations are said to form a set of overloaded functions (13.1 [over.load]). Overload resolution...